1. Introducción y Arquitectura del Sistema

El sistema **PowerLetters** está desarrollado bajo un diseño modular limpio en PHP nativo y PDO. Su objetivo es gestionar el inventario de una librería mediante operaciones **CRUD** (Crear, Leer, Actualizar y Eliminar) apoyadas por interfaces modales y relaciones de base de datos.



2. Capa de Base de Datos y Conexión (base-datos/conexion-base-datos.php)

Establece el enlace inicial con el servidor MariaDB/MySQL mediante **PDO**. Utiliza variables de configuración para el servidor, puerto, nombre de base de datos (powerletters), usuario (root) y codificación de caracteres (utf8mb4).

3. Capa de Modelo (modelo/obtener-libros.php)

Se encarga de extraer la información de la base de datos.

- **Consultas con LEFT JOIN**: Relaciona la tabla tb_libros con tb_autores, tb_generos, tb_clasificaciones y tb_editoriales para transformar los IDs numéricos en nombres de texto legibles para el usuario.

- **Datos para ComboBox**: Extrae de forma independiente los catálogos para poblar dinámicamente las listas desplegables de los modales.

4. Capa de Controladores (controlador/)

Procesa las acciones de los usuarios y ejecuta las consultas correspondientes en la base de datos:

- **guardar-libro.php**: Inserta nuevos registros en la tabla tb_libros mediante sentencias preparadas de PDO.
- **actualizar-libro.php**: Modifica los campos de un libro existente filtrando por su identificador único (id_libro).
- **eliminar-libro.php**: Borra registros de manera segura aplicando transacciones para limpiar dependencias previas (comentarios y detalles de pedidos) y evitar errores de integridad referencial (Error 1451).

5. Capa de Vista (vista/vista-prueba.php)

Renderiza la interfaz gráfica de usuario. Integra la tabla de registros, los botones de acción con iconos de lápiz (✏️) y basurero (🗑️), y los modales flotantes gemelos para el registro y la edición de libros.

6. Recursos Frontend (recursos/)

- **estilo.css**: Define los estilos visuales, colores corporativos, diseño adaptativo en rejilla (grid) y la distribución de los modales flotantes.
- **script.js**: Controla la lógica del cliente para abrir/cerrar modales y mapear automáticamente los atributos data-* de las filas de la tabla hacia los campos del formulario de edición.

7. Estructuras de Control, Manejo de Errores y Excepciones (NUEVO)

Para garantizar que el sistema sea robusto, seguro y no falle de forma imprevista, se implementaron diversas estructuras lógicas y de control de flujo. A continuación se explica **qué son**, **por qué se usaron** y **para qué sirven** en tu código:

A. Condicionales (if / else) y Operadores de Fusión

- **¿Dónde se usan?:** En los controladores (ej. if ($_SERVER["REQUEST_METHOD"] == "POST")), en las vistas (ej. if (!empty($libros)) { ... } else { ... }) y en la asignación de imágenes ($imagen = $_POST['imagen'] ?? 'default.jpg').
- **¿Para qué sirven?:** Permiten tomar decisiones lógicas en tiempo de ejecución.
- **¿Por qué se usaron?:**
    - En los controladores, el if asegura que el código de inserción o actualización **solo se ejecute si los datos realmente se enviaron por el método HTTP POST**, evitando accesos indebidos o errores por variables vacías.

    - En la vista, el if/else evalúa si la consulta devolvió resultados; si hay libros los dibuja en la tabla, y si está vacía, muestra un mensaje amigable (*"No hay libros registrados"*), evitando que la tabla colapse.
    - El operador ?? (fusión de null) sirve para verificar si el campo de imagen viene vacío; si es así, asigna automáticamente 'default.jpg', previniendo errores en la base de datos por campos nulos.

B. Manejo de Excepciones (try / catch)

- **¿Dónde se usan?:** En el archivo de conexión (conexion-base-datos.php), en el modelo (obtener-libros.php) y en todos los controladores de escritura (guardar-libro.php, actualizar-libro.php, eliminar-libro.php).
- **¿Para qué sirven?:** Capturan errores críticos (como fallos de sintaxis SQL, desconexiones del servidor o credenciales incorrectas) sin interrumpir de golpe la experiencia del usuario.
- **¿Por qué se usaron?:** PHP por defecto mostraría errores técnicos en pantalla que pueden asustar al usuario o revelar información sensible del servidor. Con try/catch, si ocurre un fallo en la base de datos, el programa lo intercepta de manera controlada y muestra un mensaje limpio o ejecuta un mecanismo de reversión.

C. Transacciones de Base de Datos (beginTransaction, commit, rollBack)

- **¿Dónde se usan?:** Exclusivamente en el archivo controlador/eliminar-libro.php.
- **¿Para qué sirven?:** Agrupan múltiples operaciones de base de datos en un "bloque atómico" (o se ejecutan todas con éxito, o ninguna).
- **¿Por qué se usaron?:** Debido a las restricciones de llaves foráneas, al intentar borrar un libro que tiene pedidos asociados, MariaDB genera un error de integridad (Error 1451). La transacción permite borrar primero los comentarios y detalles dependientes y, al final, el libro. Si el tercer paso falla, la función rollBack() deshace los borrados anteriores para evitar dejar datos huérfanos o inconsistentes en el sistema.

D. Bucles de Iteración (foreach)

- **¿Dónde se usan?:** En vista/vista-prueba.php y en modelo/obtener-libros.php.
- **¿Para qué sirven?:** Recorren de forma automática los arreglos de datos devueltos por la base de datos.
- **¿Por qué se usaron?:** Permiten transformar los resultados obtenidos en consultas PDO (fetchAll) en elementos HTML repetitivos, como las filas de la tabla (<tr>...</tr>) o las opciones de los menús desplegables (<option value="...">) de los autores, géneros y editoriales, optimizando líneas de código y automatizando la interfaz.

8. Diccionario de Datos y Variables Clave

Variable / Parámetro	Tipo	Descripción
`$_POST['id_libro']`	Entero	Identificador único del libro en operaciones de edición.
`$_POST['titulo']`	Cadena	Título o nombre principal de la obra literaria.
`$_POST['id_autor']`	Entero	Llave foránea vinculada al autor seleccionado en el ComboBox.
`$_POST['precio']`	Decimal	Valor monetario del libro con precisión de 2 decimales.
`$_POST['existencias']`	Entero	Cantidad de unidades disponibles en inventario.
`$_POST['id_genero']`	Entero	Llave foránea del género literario.
`$_POST['id_clasificacion']`	Entero	Llave foránea de la clasificación por edad/público.
`$_POST['id_editorial']`	Entero	Llave foránea de la editorial encuadernadora.

🎯 Preguntas de Entrevista de Trabajo (con respuestas)

1. Pregunta sobre Arquitectura y Diseño

> **Entrevistador:** *"Veo en tu proyecto que separaste el código en carpetas como `controlador`, `modelo`, `vista` y `recursos`. ¿Por qué decidiste estructurarlo así en lugar de tener todo en un solo archivo?"*

- **Respuesta ideal:**
  *"Decidí aplicar una arquitectura modular basada en el patrón MVC (Modelo-Vista-Controlador) para evitar el anti-patrón de 'código espagueti'. Separar las responsabilidades hace que el sistema sea escalable, mantenible y limpio: la **Vista** solo se encarga de la interfaz gráfica y presentación HTML/CSS; el **Modelo** contiene la lógica de acceso y consultas a la base de datos; los **Controladores** gestionan las peticiones HTTP y la lógica de negocio; y la carpeta **Recursos** aísla los archivos estáticos de CSS y JavaScript."*

2. Pregunta sobre Seguridad en Base de Datos

> **Entrevistador:** *"En tus controladores de inserción y actualización utilizaste PDO con `prepare()` y `execute()`. ¿Por qué usaste consultas preparadas y qué riesgo evitas con esto?"*

- **Respuesta ideal:**
  *"Utilicé consultas preparadas (Prepared Statements) con PDO para prevenir ataques de **Inyección SQL** (SQL Injection). Al usar marcadores de parámetros (como `:titulo` o `:precio`), PDO separa la estructura del comando SQL de los datos enviados por el usuario. De esta forma, cualquier entrada maliciosa introducida en un formulario se trata estrictamente como un valor literal y no como código ejecutable por MariaDB/MySQL."*

3. Pregunta sobre Integridad Referencial y Transacciones

> **Entrevistador:** *"Al momento de eliminar un libro tuviste que lidiar con claves foráneas que dependían de él (comentarios y detalles de pedidos). ¿Cómo resolviste el error 1451 de MySQL y por qué usaste una Transacción (`beginTransaction`, `commit`, `rollBack`)?"*

- **Respuesta ideal:**
  *"Para resolver la restricción de clave foránea sin romper la integridad referencial, implementé una **transacción de base de datos** en PDO. Primero se eliminan en orden los registros hijos relacionados (comentarios y detalles de pedidos) y finalmente el registro padre (el libro). Usé `beginTransaction()`, `commit()` y `rollBack()` en un bloque `try/catch` para garantizar la **atomicidad**: si por algún motivo la eliminación del libro falla a mitad del proceso, la transacción revierte todos los borrados previos (`rollBack`), evitando dejar datos corruptos o inconclusos en la base de datos."*

4. Pregunta sobre Rendimiento y Consultas SQL

Entrevistador: *"En la tabla principal muestras los nombres reales de los autores, géneros y editoriales en lugar de mostrar los IDs numéricos. ¿Cómo lograste esto en SQL y qué tipo de `JOIN` utilizaste?"*

- **Respuesta ideal:**
  "Lo logré utilizando una consulta SQL con `LEFT JOIN`. Relacioné la tabla principal `tb_libros` con las tablas de catálogo (`tb_autores`, `tb_generos`, `tb_clasificaciones`, `tb_editoriales`) mediante sus correspondientes llaves foráneas. Utilicé `LEFT JOIN` en lugar de `INNER JOIN` para asegurar que, si por alguna razón un libro no tiene asignada una editorial o un género en particular, el registro del libro se siga mostrando en la tabla sin ser descartado."

5. Pregunta sobre Frontend e Interacción JavaScript

Entrevistador: *"¿Cómo funciona la transferencia de datos desde la tabla HTML hacia el modal de edición cuando el usuario presiona el botón del lápiz?"*

- **Respuesta ideal:**
  "Aproveché los atributos HTML5 personalizados `data-*` en el botón de edición de cada fila de la tabla (como `data-titulo`, `data-precio`, etc.), poblándolos dinámicamente desde PHP. En JavaScript, al hacer clic en el botón, la función `abrirModalEditar(button)` captura esos atributos mediante `.getAttribute()` y asigna automáticamente sus valores a los campos de entrada (`input` y `select`) del formulario dentro del modal antes de cambiar su propiedad CSS `display` a `flex`."

6. Pregunta sobre Manejo de Errores y Excepciones

Entrevistador: *"¿Para qué utilizas la estructura `try ... catch` en la conexión y en los controladores PHP?"*

- **Respuesta ideal:**
  "Para la gestión de excepciones de PDO (`PDOException`). El bloque `try` intenta ejecutar el código o consulta, y si ocurre algún fallo (como credenciales incorrectas, servidor caído o error sintáctico SQL), el bloque `catch` atrapa la excepción. Esto evita que la aplicación muestre errores técnicos desestructurados que puedan poner en riesgo la seguridad del servidor o asustar al usuario final, permitiendo además registrar o mostrar mensajes de error amigables y personalizados."